Optimal Squad Selection System

An Algorithmic Approach to Constrained Combinatorial Optimization

Implementing Dynamic Programming & Branch and Bound

Usman Akram (65587)

Problem Understanding (CLO 1.1)

- **The Objective:** Maximize the total rating of a selected sports squad based on individual player ratings.
- **The Constraints:**
  - Must select *exactly* 11 players.
  - Total cost of the selected players must be *less than or equal to* a user-defined budget.
- **Algorithmic Classification:** This is a complex variation of the classic 0/1 Knapsack Problem, expanded to three dimensions to account for the fixed cardinality constraint (exactly 11 items).

Real-World Application

- **Fantasy Sports Leagues (FPL):** Millions of users globally need to select a highly constrained squad (e.g., 11 players) under a strict virtual budget.
- **Financial Portfolio Optimization:** Selecting a fixed number of assets that maximize expected return without exceeding initial capital constraints.
- **Resource Allocation & Logistics:** Assigning a strict number of personnel or vehicles to a project where cumulative salary/maintenance costs cannot surpass a departmental budget limit.

Why Use These Algorithms?

Why Dynamic Programming?

- **Guaranteed Optimality:** Unlike Greedy approaches, DP guarantees the mathematically absolute optimal squad.
- **Overlapping Subproblems:** The problem breaks down perfectly into smaller sub-budgets and sub-squads.

Why Branch & Bound?

- **Smarter Search:** It efficiently prunes branches of the state-space tree that cannot possibly yield a better result than the current best.
- **Faster than Pure Backtracking:** Reduces the exponential time drastically, making it excellent for optimization problems like team selection.

Implementation: Dynamic Programming (CLO 2.1)

- **Multi-dimensional 3D 0/1 Knapsack:** The state is tracked using a 3D table dp[n][budget][K] representing (players considered, current budget, players selected).
- **State Transitions:** For each player, we decide to either include them (adding cost and rating) or exclude them, taking the maximum of both choices.
- **Backtracking for Squad Recovery:** Simply finding the max rating isn't enough. We backtrack through the 3D dp table to recover the exact 11 players that formed the optimal solution.
- **Merge Sort:** Applied at the final stage to sort the resulting 11 players neatly for presentation to the user.

DP Algorithm & Pseudocode

```
Algorithm runDynamicProgramming(players, budget, K=11):
    Initialize 3D array dp[n+1][budget+1][K+1] with -1
     Set dp[i][w][0] = 0 // Base case: 0 players cost 0

For i = 1 to n:
    For w = 1 to budget:
        For c = 1 to K:
            dp[i][w][c] = dp[i-1][w][c] // Exclude player

             If cost[i] <= w AND dp[i-1][w - cost[i]][c-1] != -1:
                // Include player, take max
                dp[i][w][c] = MAX(dp[i][w][c], rating[i] + dp[i-1][...])

If dp[n][budget][K] == -1, Return "No valid squad"
Backtrack through dp array to find selected players
MergeSort(selected_players)
Return selected_players
```

Implementation: Branch & Bound (CLO 2.1)

- **Pre-sorting:** Players are sorted in descending order by their value-to-weight ratio (Rating / Cost). This is crucial for calculating tight upper bounds quickly.
- **State Space Tree:** A queue is utilized to explore nodes. Each node represents a decision to either include or exclude a player.
- **Upper Bound Calculation:** At any node, if the fractional upper bound of the maximum possible profit is less than the current maxProfit, the entire branch is pruned.
- **Result:** Finds the optimal squad through an intelligent, highly pruned exhaustive search.

Branch & Bound Pseudocode

```
Algorithm runBranchAndBound(players, budget, K=11):
    Sort players by (rating / cost) descending
    Initialize Queue Q, maxProfit = 0
    Enqueue empty dummy node v

    While Q is not empty:
        v = Q.dequeue()
        If v.level == n - 1, continue

        // Path 1: Include next player (u)
        u.cost = v.cost + cost[u], u.count = v.count + 1
        If u.cost <= budget AND u.count == K:
            maxProfit = MAX(maxProfit, u.profit)
        If calculateBound(u) > maxProfit AND u.count <= K:
          Q.enqueue(u)

          // Path 2: Exclude next player (u)
          u.cost = v.cost, u.count = v.count
          If calculateBound(u) > maxProfit:
          Q.enqueue(u)
```

RAM Model Analysis (CLO 3.1)

- **Underlying Assumption:** The Random Access Machine (RAM) model assumes instructions are executed sequentially and basic operations (+, -, =, <, MAX) take uniform $O(1)$ time.
- **In the context of Dynamic Programming:**
  The innermost loop of our DP approach performs constant-time checks (array lookups, addition, comparison). Therefore, the computational workload is directly proportional to the number of discrete states computed.
- **In the context of Branch & Bound:**
  The basic operations occur per visited node. The calculateBound function takes $O(K)$ basic operations, dominating the time spent at each active node in the state space tree.

Time Complexity Analysis (CLO 3.1)

Dynamic Programming

Because there are three nested loops iterating over N players, W budget, and K=11 squad size:

$$O(N \cdot W \cdot K)$$

This is pseudo-polynomial time. It is highly efficient for small budgets but scales linearly with the budget size *W*.

Branch and Bound

In the worst-case scenario (no pruning):

$$O(2^N)$$

However, due to aggressive pruning based on the fractional knapsack upper bound, the *average-case* time complexity is significantly lower. Pre-sorting takes $O(N \log N)$.

Space Complexity Analysis (CLO 3.1)

Dynamic Programming

Requires storing the entire 3-dimensional table dp[n+1][budget+1][K+1] in memory to allow for backtracking.

$$O(N \cdot W \cdot K)$$

This formulation makes the algorithm extremely memory intensive for large datasets.

Branch and Bound

Space is dictated by the size of the active Queue holding the state-space tree nodes.

$$O(2^N) \text{ Worst Case}$$

In practical application, the queue remains small due to early bounding, making B&B highly space-efficient compared to DP.

Comparison Conclusion

Metric	Dynamic Programming (3D)	Branch & Bound
Optimality	Guarantees Optimal Solution	Guarantees Optimal Solution
Time Execution	Consistent, heavily dependent on Budget	Highly variable, very fast on average
Memory Usage	Extremely High (3D Array mapping)	Low to Moderate (Queue size)
Best Use Case	Small constraints, reliable execution needed	Large datasets, optimization bottlenecks